

Chapter 1

Utilisation basique de Buildroot



Objectifs:

- Télécharger Buildroot
- Configurer un système minimal avec Buildroot pour la Raspberry Pi 3
- Lancer la compilation
- Flasher et tester le système généré
- Ajouter un paquet à une image linux

1.1 Setup

Comme spécifié dans la documentation Buildroot¹, vous avez besoin de certains paquets/dépendances :

```
sudo apt install sed make binutils gcc g++ bash patch \
gzip bzip2 perl tar cpio python unzip rsync wget libncurses-dev
```

ras ça fonctionne très bien

Lisez les spécificités de votre carte RPi3 sur différents sites notamment [le site de la Raspberry Fondation](https://www.raspberrypi.org/)

1.2 Télécharger Buildroot

Comme nous allons réaliser du développement Buildroot, on va télécharger les sources depuis le répertoire Git :

```
git clone git://git.busybox.net/buildroot
```

ou télécharger le repo sur git

¹<https://buildroot.org/downloads/manual/manual.html#requirement-mandatory>

Aller dans le nouveau dossier `buildroot` créé.
 Nous allons utiliser la branche `2026.02` de cette release.

```
cd buildroot
git checkout 2026.02 -b "2026.02"
```

ras ça fonctionne très bien

1.3 Configuration de Buildroot et génération d'une image

Vous devez maintenant trouver le fichier de configuration correspondant à votre carte. Toutes les cartes distribuées sont supportées par la version officielle de Buildroot, vous trouverez donc dans le répertoire `configs` de buildroot un fichier correspondant à votre carte. Certaines cartes incluent le nom du fabricant dans le nom du fichier de config, pensez-y !

Une fois trouvé, depuis le répertoire racine de buildroot, lancez la commande `make le_fichier_de_config` suivi de la commande `make`.

La compilation va prendre un certain temps
 cours magistral pendant que ça compile !

on va dans le dossier `configs` on recopie le nom du fichier config. on revient au dossier parent de `configs`
`cd` Quand vous en êtes là, dites le moi et on continuera le
`make nom_fichier`
`make`

Une fois la compilation terminée, rendez-vous dans le répertoire `output/images`

Les cartes fournies sont capables de démarrer sur une carte micro-SD. Le nom du fichier de sortie est souvent `sdcard.img`, il contient une image complète incluant un bootloader, le noyau, le device-tree ainsi qu'un rootfs.

Il vous faut maintenant copier ce binaire sur la carte micro-SD fournie. Vous aurez besoin des droits root (via `sudo`) pour cela.

cela prendra du temps et demande une bonne connexion internet
 à la fin, un fichier `sdcard.img` sera créé dans le dossier `output/images`

Attention ! Lisez bien les instructions, la commande suivante peut endommager votre installation Linux si les mauvais paramètres sont passés ! N'hésitez pas à demander de vérifier !

Lorsque vous brancherez la carte micro-SD sur votre PC (soit telle-quelle, soit avec un adaptateur), un nouveau noeud sera créé. Plusieurs noms sont possibles :

- `/dev/sdX` **Attention, il y a de TRÈS grandes chances que `/dev/sda` soit votre disque dur, et non la carte SD !**
 - `/dev/mmcblkX`
- dans notre cas, c'est `/dev/sdb`

La meilleure manière de ne pas faire d'erreur est de comparer le contenu de votre répertoire `/dev` avant et après avoir connecté la carte SD.

Pour peupler la carte SD, nous allons faire une copie brute de données, grâce à la commande `dd` :

(Ici, je prends l'exemple avec votre carte SD sur `/dev/mmcblk0`). On va s'assurer que la carte SD n'est pas montée automatiquement comme on veut tout enlever.

```
sudo umount /dev/mmcblk0*
```

donc on fait : `sudo umount /dev/sdb*`

```
sudo dd if=output/images/<fichier_image> of=/dev/XXX bs=1M conv=fdatasync
pas besoin de mettre l'adresse complète du fichier
```

Vous pouvez maintenant débrancher la carte SD, et la mettre dans votre carte !

```
sudo dd if=sdcard.img of=/dev/sdb bs=1M conv=fdatasync
```

1.4 Connexion à la liaison série

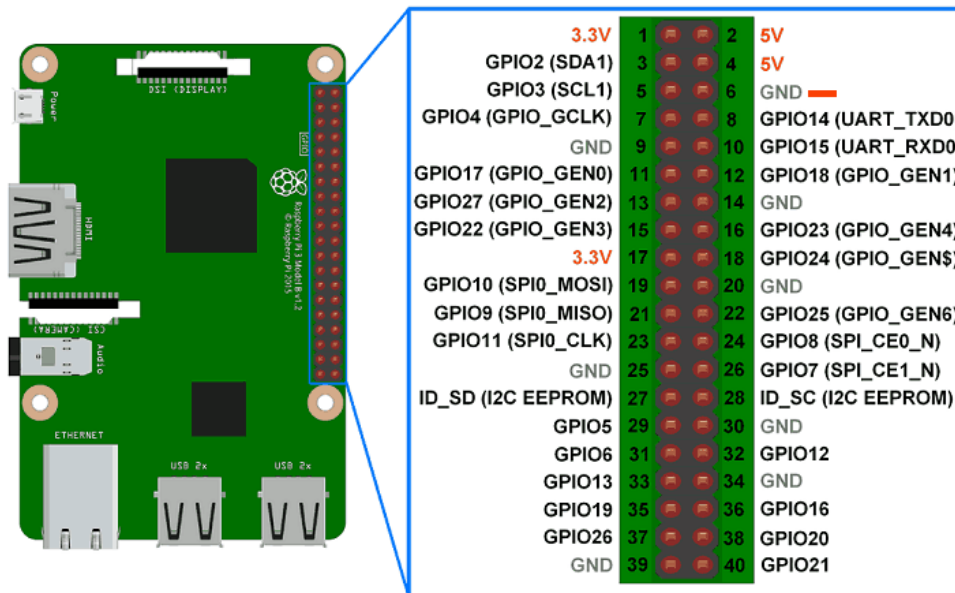
Pour ce TP, vous aurez une Raspberry Pi. Beaucoup de systèmes embarqués conçus pour faire tourner Linux ont cependant des similarités.

La première est la présence quasi systématique d'une **liaison série** utilisant le protocole RS-232. Elle permet d'accéder à un terminal, souvent avec les droits `root`.

Pour se connecter au lien série, nous avons besoin d'un adaptateur série vers USB, bien que certaines cartes intègrent un adaptateur et permettent de se connecter directement en USB. Dans tous les cas, vous devriez avoir reçu l'adaptateur qui va bien.

Pour les adaptateurs RS232 vers USB, la convention souvent utilisée est :

- Fil Bleu / Noir : GND
- Fil Vert : RX
- Fil Rouge : TX



bleu - vert - rouge
de la droite vers la
gauche

Sur les fiche données, vous trouverez l'emplacement des signaux **RX** et **TX** de chaque carte, mais n'oubliez pas qu'il vous faut croiser les signaux pour que l'adaptateur fonctionne ! Ainsi, il vous faut brancher le **RX** de l'adaptateur sur le **TX** de la carte, et le **TX** de l'adaptateur sur le **RX** de la carte !

Une fois branché, vérifiez qu'il est bien détecté sur votre PC :

```
ls /dev
```

Vous devriez voir un fichier nommé `/dev/ttyUSB0` ou bien `/dev/ttyACM0`.

Pour vous connecter à la console série, il faut utiliser un **émulateur de terminal**. Il en existe plusieurs, tels que **screen**, **minicom** ou encore **picocom**.

Installez **picocom** sur votre système : `sudo apt install picocom`.

Vous pouvez ensuite vous connecter à votre board :

```
sudo picocom -b 115200 /dev/ttyUSB0 (or /dev/ttyACM0)
```

Vous voilà connecté à votre board !

RAS on fait ces deux commandes tout marche on verra normalement terminal ready et on verra le login de buildroot faut que la carte soit connectée et carte sd insérée

1.5 Démarrage du système

Insérez la carte SD dans la RPi3.

Allumez via USB votre board et vous devriez voir le système booter !

Le login est **root** et profitez pour explorer le système. Lancez **ps** pour regarder combien de processus sont en cours d'exécution, qu'est-ce que Buildroot a généré dans `/bin`, `/lib`, `/usr` et `/etc`.

BRAVO ! Vous avez compilé votre première image pour une Raspberry Pi avec Buildroot ! C'est plus satisfaisant que juste télécharger une image déjà prête, non ? :D

1.6 Ajouter un paquet à l'image

Maintenant que vous êtes en mesure de créer votre propre image pour votre système, il est temps de la personnaliser un peu.

La configuration dans Buildroot se fait via la commande `make menuconfig`

ceci doit être fait en étant root car on a tout fait en

Naviguez dans **System Configuration** -> **System banner**, et changez la valeur par défaut par celle de votre choix :)

étant root, on le fait sur l'ordi et non sur le raspberry

Allez ensuite dans **Target Packages** -> **Games** et sélectionnez le paquet `ascii_invaders`

Recompilez votre image avec la commande **make**, re-flashez là sur votre carte micro-SD et redémarrez. vous devriez pouvoir jouer à `ascii-invaders` ! :D

Chapter 2

Création d'un nouveau paquet dans Buildroot

Objectifs:

- Créer d'un nouveau paquet pour *tint*
- Comprendre comment ajouter des dépendances

2.1 Paquet minimal

Créer un répertoire pour ce paquet dans les sources de Buildroot : `package/tint`. Créer un fichier `Config.in` avec une option pour activer ce paquet et un fichier `tint.mk` minimal qui spécifiera ce qu'il faut pour juste télécharger le paquet.

Pour information, l'URL de téléchargement des sources *tint* est <https://github.com/DavidGriffith/tint.git>.

Note: Pour y arriver, seulement deux variables sont nécessaires d'être définies dans le fichier `.mk` ainsi qu'un appel à la macro de l'infrastructure appropriée.

Maintenant, allez dans le `menuconfig`, activez *tint*, et lancez une compilation `make`. Vous devriez voir l'archive *tint* être téléchargée et extraite. Regardez dans `output/build/` pour voir si ça a été extrait comme attendu.

2.2 C'est parti pour la compilation !

tint utilise un simple `Makefile` pour son processus de build. Vous allez donc devoir définir les variables de build pour *tint*. Pour faire cela, 4 variables sont fournies par Buildroot :

- `TARGET_MAKE_ENV`, qui devra être passé dans l'environnement utilisé lors de la commande `make`.
- `MAKE`, contient le vrai appel du `make` avec potentiellement quelques paramètres pour paralléliser le build.
- `TARGET_CONFIGURE_OPTS`, contient la définition de pleins de variables utiles dans les `Makefiles` comme : `CC`, `CFLAGS`, `LDFLAGS`, etc. Et notamment les options pour cross-compiler !

- `@D`, contient le chemin vers le répertoire où le code source de *tint* a été extrait.

Quand vous créez des paquets Buildroot, c'est une bonne pratique de regarder comment les autres paquets sont définis. Regardez par exemple le paquet `jhead` qui sera très similaire à ce dont on a besoin pour notre paquet `tint`.

Lorsque vous avez écrit l'étape de build de *tint*, il est temps de tester ! Mais si vous lancez seulement un `make` pour redémarrer un build, le paquet `tint` ne sera pas rebuildé car il l'a été déjà été précédemment.

On va donc forcer le build en supprimant complètement le répertoire des sources :

```
make tint-dirclean
```

Ensuite, recommencez un build :

```
make
```

Cette fois, vous devriez voir l'étape `tint f16955649eb7968980449af586d8ec213bb43922 Building`.

2.3 Gérer les dépendances

Il se peut que vous deviez gérer les dépendances. `tint` dépend de la bibliothèque `ncurses` pour l'interface sur un terminal en mode "texte". On va donc installer `ncurses` comme dépendance de *tint*. Pour cela :

- Indiquez la dépendance dans le fichier `Config.in`. Utilisez le `select` pour être sûr que le paquet `ncurses` sera automatiquement sélectionné avec la sélection de `tint`.
- Indiquez la dépendance dans le fichier `.mk`.

Recommencez un build du paquet avec : `make tint-dirclean all` (qui est pareil que `make tint-dirclean` suivi d'un `make`).

Maintenant, le paquet devrait être compilé avec succès ! Si vous jetez un oeil à `output/build/tint-.../`, vous devriez voir le binaire `tint`. Lancer le programme `file` sur `tint` pour vérifier que c'est bien pour ARM. `tint` a bien été compilé mais il n'est pas pour autant installé dans le rootfs de notre cible !

2.4 Installation et test du programme

Si vous regardez le `Makefile` des sources de *tint*, vous remarquez qu'il n'y a rien pour installer le programme. Il n'y a pas de règle `install`.

Dans `tint.mk`, on va devoir créer une *commande d'installation pour cible* qui va simplement installer le binaire `tint`. Utilisez la variable `$(INSTALL)` pour cela. Regardez le paquet `jhead` comme exemple.

RebUILdez une nouvelle fois `tint` et cette fois, vous devriez voir le binaire `tint` dans `output/target/usr/bin/` !

Reflashez votre rootfs sur la carte SD et rebooter le système. Il est un peu bizarre ce jeu tetris, non ?

2.5 Résolution du problème du jeu

Si vous testez le jeu sur votre ordinateur, vous verrez que les pièces sont de couleurs. Sur la RPi, on ne voit aucune pièce du tétis.

C'est un problème de terminal ! En effet, le terminal utilisé par défaut par Buildroot est `vt100` qui n'a pas de support pour la couleur !

Recherchez dans le `menuconfig` de buildroot, quelle variable vous devez changer pour que `GETTY` devienne un terminal qui supporte la couleur (et trouvez lequel !).

Retestez pour être sûr que tout fonctionne :)

2.6 Ajout d'un fichier de hash

Pour finaliser notre paquet, ajoutez un fichier manquant : *hash file* pour permettre de vérifier que les personnes builderont les mêmes sources que celles que vous avez utilisé pour votre paquet. Pour savoir le hash, vous pouvez directement l'avoir depuis les sources que vous avez téléchargé depuis Buildroot. Pour cela, on utilisera `sha256sum` et le dossier `dl` de Buildroot :

```
sha256sum dl/tint/tint-f3f757f6f9b79b1fd70c37a46ac4bbe2dc76f30b-git4.tar.gz
```

Inspirez-vous de `jhead` et le fichier `hash`. Une fois que vous pensez que c'est bon, on recompile avec `make tint-dirclean all`. Regarder l'output du build et avant le message `Extracting`, vous devriez voir :

```
tint-f3f757f6f9b79b1fd70c37a46ac4bbe2dc76f30b-git4.tar.gz: OK (sha256: 275f65b5a0975231750d9d52a40a61d3e
```

2.7 Amélioration de votre paquet

Si ce n'est pas déjà fait, améliorez le paquet de `tint` pour gérer son fichier de score, par exemple. Actuellement, il n'est pas créé et n'est donc pas disponible pour enregistrer les résultats ! C'est quand même dommage surtout qu'on aura besoin de `/var/games/` pour le prochain lab !

2.8 Test de la suppression d'un paquet

Pour jouer un peu avec Buildroot, réaliser les tests suivants : désactiver le paquet `tint` dans le `menuconfig` et redémarrer le build via `make`. Quand le build est fini, regarder dans `output/target/`. Est-ce que `tint` est toujours installé ? Si oui, pourquoi ?

Chapter 3

Utilisation de git pour appliquer du code et logger les infos du jeu

Objectifs:

- Récupération des sources de *tint*
- Application d'un patch
- Compilation et test sur le PC hôte
- Modification du paquet *tint*

3.1 Préparation

Après une recherche Google, trouvez le site web pour *tint* et téléchargez le code source avec *git*, en dehors de buildroot.

```
git clone https://github.com/DavidGriffith/tint.git.
```

Je vous enverrai un patch qui vous permettra d'avoir une version minimaliste des modifications sur le code du jeu pour récupérer des données !

3.2 Application d'un patch

Allez dans le répertoire git de *tint* et appliquez le patch

```
git am <nom_du_patch.patch
```

Cela va appliquer mes modifications sur le code que vous venez de télécharger. Familiarisez vous avec les commandes git si vous n'en avez pas l'habitude :

```
git show
```


3.3 Compilation pour X86

Juste pour vous faire voir une compilation d'un logiciel C ainsi que les différences entre les architectures (x86 vs ARM), on va compiler pour notre PC de développement. Dans le dossier du code de *tint* :

```
make
```

Normalement, tout compilera comme il faut. MAIS pour votre ordinateur et non pas pour la RPi. Vérifiez ça avec :

```
file tint
```

Comparez la sortie de cette commande avec le *tint* qui a été compilé par Buildroot et donc dans `buildroot/output/target/usr/bin/tint`

Comme vous l'avez compilé pour votre PC, vous pouvez le lancer et tester si le log est bien créé ! Alors, faites quelques parties et profitez en pour vous familiariser avec le code, les modifications, ...

3.4 Exporter les modifications dans Buildroot

Une fois que tout est bon, exportez les patches dans buildroot pour qu'ils soient appliqués sur *tint* pour la RPi.

Pour cela, rien de plus simple ! Buildroot se charge automatiquement d'appliquer tous les "fichier.patch" qui sont dans le même dossier que le Makefile *tint.mk*.

Faites les modifications nécessaires, re-compilez une image et testez vos modifications. Si vous avez un problème de permissions, regardez comment gérer !

3.5 Améliorer le log

Maintenant que vous avez compris comment modifier le code *tint* et l'installer sur la RPi, vous pouvez tester de modifier le code pour ajouter toutes les informations que vous pourriez avoir.

Les logs sont exploitables qu'une fois le jeu terminé. Ça serait tellement cool de pouvoir voir en direct les informations du log, non ? (Petit tips, regardez avec `tail -f` ou `inotify` !

Vous pouvez choisir le langage que vous voulez si vous préférez en python ! Par contre, il faudra penser à ajouter le support de Python sur la RPi.